

✓ CSA6502 – Generative AI and LLMs

Lab Experiments 15–24: Embeddings, Vector Search & RAG

Name: Joannathan Packia Singh | **Reg. No.:** 192472229 | **Course:** CSA6502

Lab	Task
15	Generate text embeddings and perform semantic similarity search
16	Semantic search system using cosine similarity
17	Build a vector database (ChromaDB) and retrieve documents
18	Document storage and top-k retrieval using a vector database
19	Document Question-Answering using RAG
20	End-to-end RAG pipeline: load → chunk → embed → retrieve → generate
21	Domain-specific chatbot using LangChain + a vector database
22	Context-aware chatbot using LangChain, retrieval, and an LLM
23	Simple AI assistant answering questions using external documents
24	Document-based AI assistant supporting multiple uploaded documents

✓ Setup — install packages and load shared resources

Run this once before any lab below. Restart the runtime once after this cell if you've run an older version of this notebook in the same session (old package versions may already be imported).

```
# Pinned installs.
# Colab's default `pip install langchain langchain-community ...` (no versions) currently
# resolves to LangChain 1.x, which REMOVES `langchain.chains` (RetrievalQA,
# ConversationalRetrievalChain) and `langchain.memory` entirely – that's exactly why the
# Lab 21/22 cells were breaking with ModuleNotFoundError. Pinning to the 0.3.x line keeps
# the classic chain APIs this notebook (and most course material) is written against.
!pip install -q --upgrade \
    "langchain==0.3.27" "langchain-community==0.3.27" "langchain-groq==0.3.8" \
    "langchain-chroma==0.2.6" "langchain-huggingface==0.3.1" \
    sentence-transformers chromadb groq pypdf pysqlite3-binary
```

```
# Colab's system sqlite3 is often older than what ChromaDB requires, which raises
# "RuntimeError: Your system has an unsupported version of sqlite3" the moment chromadb
# is imported anywhere below. Swap in pysqlite3-binary BEFORE that happens. Safe to run
# even where it isn't needed (falls through silently).
```

```
try:
    __import__("pysqlite3")
    import sys
    sys.modules["sqlite3"] = sys.modules.pop("pysqlite3")
except ImportError:
    pass
```

```
from google.colab import userdata
from sentence_transformers import SentenceTransformer, util
from groq import Groq
import numpy as np
```

```
GROQ_API_KEY = userdata.get('groq')
if not GROQ_API_KEY:
    raise RuntimeError(
        "No Groq key found. In Colab: left sidebar -> key icon -> 'Add new secret' -> "
        "name it 'groq' -> paste your Groq API key -> toggle notebook access on."
    )
```

```
groq_client = Groq(api_key=GROQ_API_KEY)
LLM_MODEL = "openai/gpt-oss-120b"
```

```
# Shared embedding model used across every lab below
embedder = SentenceTransformer('all-MiniLM-L6-v2')
```

```
def ask_llm(prompt, model=LLM_MODEL, temperature=0.2):
    """Simple helper: send a prompt to Groq and return the text answer."""
    response = groq_client.chat.completions.create(
        model=model,
        messages=[{"role": "user", "content": prompt}],
        temperature=temperature,
    )
    return response.choices[0].message.content
```

```
print("Embedding model and Groq client ready.")
```

```
# Shared sample document set used in Labs 15-19
documents = [
    "The mitochondria is the powerhouse of the cell, generating ATP through respiration.",
    "Photosynthesis converts sunlight, water, and carbon dioxide into glucose and oxygen.",
    "Python is a high-level, interpreted programming language known for readability.",
    "The French Revolution began in 1789 and led to the end of the monarchy.",
    "Neural networks are composed of layers of interconnected nodes called neurons.",
    "The Great Barrier Reef is the world's largest coral reef system, located off Australia.",
    "Machine learning models improve their performance by learning patterns from data.",
    "The human heart has four chambers: two atria and two ventricles.",
    "Blockchain is a distributed ledger technology used to record transactions securely.",
    "The Amazon rainforest produces roughly 20% of the world's oxygen supply.",
]
print(f"{len(documents)} sample documents loaded.")
```

10 sample documents loaded.

Lab 15 — Generate Text Embeddings and Perform Semantic Similarity Search

Goal: Convert text into numeric embeddings and find the most semantically similar text to a query — without any vector database, just raw embeddings.

```
query = "How do plants make their own food?"

# Embed the query and all documents
query_embedding = embedder.encode(query, convert_to_tensor=True)
doc_embeddings = embedder.encode(documents, convert_to_tensor=True)

# Cosine similarity between query and every document
similarities = util.cos_sim(query_embedding, doc_embeddings)[0]

# Rank and show top 3
top_results = np.argsort(-similarities.numpy())[:3]
print(f"Query: {query}\n")
for idx in top_results:
    print(f"Score: {similarities[idx]:.4f} | {documents[idx]}")

# --- For understanding: cosine similarity is just the normalized dot product ---
# similarity = (A . B) / (||A|| * ||B||). This confirms util.cos_sim isn't a black box.
q_vec = query_embedding.numpy()
best_doc_vec = doc_embeddings.numpy()[top_results[0]]
manual_cosine = np.dot(q_vec, best_doc_vec) / (np.linalg.norm(q_vec) * np.linalg.norm(best_doc_vec))
print(f"\nManual cosine similarity for the top match (sanity check): {manual_cosine:.4f}")
```

Query: How do plants make their own food?

Score: 0.3927 | Photosynthesis converts sunlight, water, and carbon dioxide into glucose and oxygen.
Score: 0.2664 | The mitochondria is the powerhouse of the cell, generating ATP through respiration.
Score: 0.2098 | The Amazon rainforest produces roughly 20% of the world's oxygen supply.

Manual cosine similarity for the top match (sanity check): 0.3927

Lab 16 — Semantic Search System Using Cosine Similarity (Document vs Query)

Goal: Wrap the logic from Lab 15 into a reusable semantic search function that takes any query and returns ranked documents with scores.

```
def semantic_search(query, docs, top_k=3):
    q_emb = embedder.encode(query, convert_to_tensor=True)
    d_emb = embedder.encode(docs, convert_to_tensor=True)
    scores = util.cos_sim(q_emb, d_emb)[0].numpy()
    ranked_idx = np.argsort(-scores)[:top_k]
    return [(docs[i], float(scores[i])) for i in ranked_idx]

# Try it with a different query
results = semantic_search("What is used to record transactions securely?", documents, top_k=3)
for doc, score in results:
    print(f"{score:.4f} | {doc}")
```

0.5874 | Blockchain is a distributed ledger technology used to record transactions securely.
0.0900 | Python is a high-level, interpreted programming language known for readability.
0.0812 | Machine learning models improve their performance by learning patterns from data.

Lab 17 — Build a Vector Database (ChromaDB) and Perform Similarity-Based Retrieval

Goal: Instead of computing cosine similarity manually every time, store the documents in a proper vector database (ChromaDB) and let it handle indexing and retrieval.

```
import chromadb
from chromadb.utils import embedding_functions

# ChromaDB needs an embedding function wrapper around our sentence-transformers model
sentence_transformer_ef = embedding_functions.SentenceTransformerEmbeddingFunction(
    model_name="all-MiniLM-L6-v2"
)

chroma_client = chromadb.Client()
collection = chroma_client.get_or_create_collection(
    name="lab_documents",
    embedding_function=sentence_transformer_ef,
    # ChromaDB defaults to squared-L2 distance. We force cosine so the numbers below are
    # directly comparable to the cosine similarity scores from Labs 15-16.
    metadata={"hnsw:space": "cosine"},
)

# upsert (not add): re-running this cell won't throw a "duplicate ID" error
collection.upsert(
    documents=documents,
    ids=[f"doc_{i}" for i in range(len(documents))],
)

# Query the vector DB directly
results = collection.query(query_texts=["Tell me about the structure of the human heart"], n_results=3)
for doc, dist in zip(results["documents"][0], results["distances"][0]):
    similarity = 1 - dist # cosine distance -> cosine similarity
    print(f"Similarity: {similarity:.4f} | {doc}")
```

```
Similarity: 0.5867 | The human heart has four chambers: two atria and two ventricles.
Similarity: 0.2350 | The mitochondria is the powerhouse of the cell, generating ATP through respiration.
Similarity: 0.1811 | Deep learning uses multi-layered neural networks to model complex patterns.
```

Lab 18 — Implement Document Storage and Top-K Retrieval Using a Vector Database

Goal: Extend Lab 17 with a larger document store and a reusable top-k retrieval function that returns clean results with scores.

```
# A slightly larger store, reusing the same collection from Lab 17
extra_documents = [
    "Cloud computing delivers computing services like storage and servers over the internet.",
    "The Pacific Ocean is the largest and deepest of Earth's oceans.",
    "Deep learning uses multi-layered neural networks to model complex patterns.",
    "The Renaissance was a period of cultural rebirth in Europe from the 14th to 17th century.",
]
collection.upsert(
    documents=extra_documents,
    ids=[f"doc_{i}" for i in range(len(documents), len(documents) + len(extra_documents))],
)

def retrieve_top_k(query, k=3):
    results = collection.query(query_texts=[query], n_results=k)
    docs = results["documents"][0]
    similarities = [1 - d for d in results["distances"][0]]
    return list(zip(docs, similarities))

for doc, sim in retrieve_top_k("What technology stores transactions across many computers?", k=3):
    print(f"Similarity: {sim:.4f} | {doc}")
```

```
Similarity: 0.4334 | Blockchain is a distributed ledger technology used to record transactions securely.
Similarity: 0.3956 | Cloud computing delivers computing services like storage and servers over the internet.
Similarity: 0.1715 | Machine learning models improve their performance by learning patterns from data.
```

Lab 19 — Develop a Document Question-Answering System Using RAG

Goal: Combine retrieval (Lab 18) with an LLM (Groq) — retrieve the most relevant chunks, then ask the LLM to answer strictly using that retrieved context. This is the core RAG pattern.

```
def rag_answer(question, k=3):
    retrieved = retrieve_top_k(question, k=k)
    context = "\n".join([f"- {doc}" for doc, _ in retrieved])

    prompt = f"""Answer the question using ONLY the context below.
```

If the context does not contain the answer, say "I don't have enough information."

```
Context:
{context}
```

```
Question: {question}
Answer: ""
```

```
# Explicitly pass a working model name to override the problematic LLM_MODEL global
return ask_llm(prompt, model="openai/gpt-oss-120b"), retrieved
```

```
question = "What percentage of the world's oxygen does the Amazon rainforest produce?"
answer, sources = rag_answer(question)
```

```
print("Answer:", answer)
print("\nRetrieved context used:")
for doc, sim in sources:
    print(f"  - ({sim:.4f}) {doc}")
```

Answer: 20%

Retrieved context used:

- (0.8811) The Amazon rainforest produces roughly 20% of the world's oxygen supply.
- (0.4010) Photosynthesis converts sunlight, water, and carbon dioxide into glucose and oxygen.
- (0.2707) The Pacific Ocean is the largest and deepest of Earth's oceans.

✓ Lab 20 — Build an End-to-End RAG Pipeline (Load → Chunk → Embed → Retrieve → Generate)

Goal: Formalize Labs 15-19 into one complete pipeline that works on a real document: load a file, split it into chunks, embed and store the chunks, retrieve relevant ones, and generate an answer.

Note: point `uploaded_filename` below at your own PDF in Drive if you have one. If it isn't found (e.g. Drive isn't mounted, or the path is wrong), the cell automatically falls back to a small auto-generated sample document about RAG itself — so the rest of the pipeline always has something real to run on instead of crashing on a missing file.

```
import os

# Point this at your own file in Drive if you have one – right-click the file in Drive ->
# "File information" -> "Copy path". If it's not found, a sample document is auto-generated
# below so the lab still runs end-to-end.
uploaded_filename = "/content/drive/MyDrive/1.pdf" # <-- optionally update this path

drive_available = False
try:
    from google.colab import drive
    drive.mount('/content/drive', force_remount=False)
    drive_available = True
except Exception as e:
    print(f"Drive mount skipped/unavailable ({e}); will use the fallback sample document.")

if drive_available and os.path.exists(uploaded_filename):
    print(f"Using your file: {uploaded_filename}")
else:
    uploaded_filename = "/content/sample_doc.txt"
    sample_text = """Retrieval-Augmented Generation, or RAG, is a technique that combines two
components: a retriever and a generator. Instead of relying only on what a language model
memorized during training, RAG first searches an external knowledge source for the most
relevant pieces of text, then hands those pieces to the language model as context before
it produces an answer. This keeps answers grounded in real, up-to-date documents rather
than the model's frozen internal knowledge, and it drastically reduces hallucination on
domain-specific questions.
```

The retriever in a RAG system is usually powered by text embeddings. An embedding model converts a chunk of text into a dense numeric vector such that semantically similar pieces of text end up close together in vector space. A query is embedded the same way, and the system compares the query vector against every stored document vector using a similarity measure such as cosine similarity, returning the closest matches.

Storing and searching millions of these vectors efficiently is the job of a vector database such as ChromaDB or FAISS. These systems build specialized indexes (for example HNSW graphs) so that finding the top-k nearest vectors to a query takes milliseconds instead of scanning every document one by one.

Before embedding, long documents are split into smaller chunks. Chunking matters because embedding models have a limited context window, and because smaller, focused chunks tend to retrieve more precisely than one giant embedding for an entire document. A common strategy is a recursive character splitter with some overlap between consecutive chunks, so that a sentence split across a chunk boundary is not lost entirely from either chunk.

Once relevant chunks are retrieved, they are inserted into a prompt template instructing the language model to answer strictly from the provided context and to admit when the context does not contain the answer. This final generation step is what turns a plain search engine into a conversational question-answering assistant, and it is the same core pattern used by LangChain's RetrievalQA and ConversationalRetrievalChain chains later in this notebook."""

```
with open(uploaded_filename, "w") as f:
```

```
f.write(sample_text)
print(f"No Drive file found – using auto-generated sample document: {uploaded_filename}")
```

Mounted at /content/drive
No Drive file found – using auto-generated sample document: /content/sample_doc.txt

```
# Imported from the standalone `langchain_text_splitters` package rather than
# `langchain.text_splitter` – that submodule was removed from newer LangChain releases,
# but the standalone package works regardless of which LangChain version is installed.
from langchain_text_splitters import RecursiveCharacterTextSplitter

def load_text(path):
    if path.lower().endswith(".pdf"):
        from pypdf import PdfReader
        reader = PdfReader(path)
        return "\n".join(page.extract_text() or "" for page in reader.pages)
    else:
        with open(path, "r", encoding="utf-8", errors="ignore") as f:
            return f.read()

raw_text = load_text(uploaded_filename)

splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
chunks = splitter.split_text(raw_text)
print(f"Document split into {len(chunks)} chunks.")
print("\nFirst chunk preview:\n", chunks[0][:300])
```

Document split into 6 chunks.

First chunk preview:
Retrieval-Augmented Generation, or RAG, is a technique that combines two components: a retriever and a generator. Instead of relying only on what a language model memorized during training, RAG first searches an external knowledge source for the most relevant pieces of text, then hands those pieces

```
# Store chunks in their own ChromaDB collection
pipeline_collection = chroma_client.get_or_create_collection(
    name="pipeline_docs",
    embedding_function=sentence_transformer_ef,
    metadata={"hnsw:space": "cosine"},
)
pipeline_collection.upsert(
    documents=chunks,
    ids=[f"chunk_{i}" for i in range(len(chunks))],
)

def full_rag_pipeline(question, k=3):
    results = pipeline_collection.query(query_texts=[question], n_results=k)
    retrieved_chunks = results["documents"][0]
    context = "\n\n".join(retrieved_chunks)

    prompt = f"""Answer the question using ONLY the context below.
    If the answer isn't in the context, say so.

    Context:
    {context}

    Question: {question}
    Answer: ""
    """
    return ask_llm(prompt, retrieved_chunks)

# Try asking a question about the uploaded (or auto-generated) document
question = "Summarize the main topic of this document."
answer, chunks_used = full_rag_pipeline(question)
print("Answer:\n", answer)
```

Answer:
The document discusses Retrieval-Augmented Generation (RAG), explaining how it combines a retriever that fetches relevant text chunks from

✓ Lab 21 — Develop a Domain-Specific Chatbot Using LangChain and a Vector Database

Goal: Wrap the same retrieve-then-generate pattern in LangChain's own abstractions (`Chroma` vectorstore + `RetrievalQA` chain) instead of writing the retrieval and prompting manually.

This was the cell that was failing before: with an unpinned `pip install langchain ...`, Colab installs LangChain 1.x, and `from langchain.chains import RetrievalQA` raises `ModuleNotFoundError: No module named 'langchain.chains'` because that whole module was removed in the 1.x rewrite. The pinned install in the Setup cell (`langchain==0.3.27`) restores it.

```
from langchain_chroma import Chroma
from langchain_huggingface import HuggingFaceEmbeddings
```

```

from langchain.groq import ChatGroq
from langchain.chains import RetrievalQA

# LangChain-native embeddings wrapper (same underlying model)
lc_embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")

# Build a LangChain Chroma vectorstore from the chunks created in Lab 20
lc_vectorstore = Chroma.from_texts(
    texts=chunks,
    embedding=lc_embeddings,
    collection_name="langchain_domain_bot",
)

lc_llm = ChatGroq(groq_api_key=GROQ_API_KEY, model_name=LLM_MODEL, temperature=0.2)

domain_qa_chain = RetrievalQA.from_chain_type(
    llm=lc_llm,
    retriever=lc_vectorstore.as_retriever(search_kwargs={"k": 3}),
    return_source_documents=True,
)

response = domain_qa_chain.invoke({"query": "What is this document mainly about?"})
print("Answer:", response["result"])

```

Loading weights: 100%

103/103 [00:00<00:00, 2204.44it/s]

Answer: The passage explains **Retrieval-Augmented Generation (RAG)**—how it works by first retrieving relevant text chunks from an external

Lab 22 — Implement a Context-Aware Chatbot Using LangChain, Retrieval, and an LLM

Goal: Extend Lab 21 with conversation memory so the chatbot understands follow-up questions that depend on earlier turns (e.g. "can you explain that more simply?" after asking about a document).

```

from langchain.chains import ConversationalRetrievalChain
from langchain.memory import ConversationBufferMemory

memory = ConversationBufferMemory(memory_key="chat_history", return_messages=True)

context_chatbot = ConversationalRetrievalChain.from_llm(
    llm=lc_llm,
    retriever=lc_vectorstore.as_retriever(search_kwargs={"k": 3}),
    memory=memory,
)

# First turn
r1 = context_chatbot.invoke({"question": "What is the document about?"})
print("Bot:", r1["answer"])

# Follow-up turn that relies on the previous turn's context (tests the memory)
r2 = context_chatbot.invoke({"question": "Can you explain that in simpler terms?"})
print("\nBot:", r2["answer"])

```

/tmp/ipykernel_722/1370807395.py:4: LangChainDeprecationWarning: Please see the migration guide at: <https://python.langchain.com/docs/versions>

memory = ConversationBufferMemory(memory_key="chat_history", return_messages=True)

Bot: The document explains **Retrieval-Augmented Generation (RAG)**—a method that pairs a retriever with a language-model generator so that

Bot: **Retrieval-Augmented Generation (RAG)** in plain language

- Two-step process**
 - Retriever**: First the system looks for the most relevant pieces of text in an external source (like a set of documents, a knowledge base, or a database).
 - Generator**: Then it feeds those pieces to a language model, which writes the answer **using only that supplied context**.

- Why it matters**
 - The language model doesn't have to "remember" everything it ever saw during training.
 - It can give answers that are up-to-date and grounded in real documents, because the retrieved text is taken right from the source.

Chunking – breaking big documents into smaller pieces

- What is chunking?**
Before the retriever can search, long documents are split into many short "chunks" (e.g., a few sentences or a paragraph each).

- Why split them?**
 - Size limits** – Embedding models (the part that turns text into searchable vectors) can only handle a limited amount of text at once.
 - Precision** – Small, focused chunks make it easier to find exactly the part that matches the question. A single huge chunk would miss relevant details.

- How it's done**
A common method is a *recursive character splitter* that creates overlapping chunks. Overlap ensures that a sentence that straddles a boundary is still captured in a chunk.

Putting it together

- Chunk the documents** → each chunk gets its own embedding (a numeric fingerprint).

2. ****When a question arrives****, the retriever loops through the question's embedding to all chunk embeddings and picks the most relevant ones.
3. ****Those chunks are placed into a prompt**** that tells the language model: "Answer only using this text, and say you don't know if the answer is not in the text."
4. ****The generator writes the final answer****, giving you a response that is both fluent and anchored in real, up-to-date information.

So, RAG = "search-then-write," and chunking is the essential preprocessing step that makes the search accurate and efficient.

Lab 23 — Develop a Simple AI Assistant Capable of Answering Questions Using External Documents

Goal: Turn the pipeline from Lab 20 into an interactive loop — a minimal general-purpose assistant that keeps answering questions about the uploaded document until you stop it.

Note: if you run this notebook non-interactively (e.g. "Run all"), there's no keyboard to type into — the cell below detects that and falls back to a couple of demo questions instead of hanging or crashing.

```
def simple_assistant(num_turns=3, demo_questions=None):
    print("Simple document assistant ready. Type your questions below (type 'exit' to stop early).")
    demo_questions = demo_questions or ["What is this document about?", "exit"]
    for i in range(num_turns):
        try:
            q = input("\nYou: ")
        except (EOFError, OSError):
            # No interactive stdin available (e.g. Run All) -> fall back to a demo question
            q = demo_questions[min(i, len(demo_questions) - 1)]
            print(f"You (auto-demo, no interactive input available): {q}")
        if q.lower() in ("exit", "quit", ""):
            break
        answer, _ = full_rag_pipeline(q)
        print(f"Assistant: {answer}")

simple_assistant(num_turns=3)
```

Simple document assistant ready. Type your questions below (type 'exit' to stop early).

You: What is this document about?

Assistant: The document explains Retrieval-Augmented Generation (RAG)—a method that combines a retriever (typically using text embeddings to find relevant information in a document) with a generator (which uses the retrieved information to produce a coherent and contextually relevant answer).

You: Summarize the document

Assistant: The document explains Retrieval-Augmented Generation (RAG), which pairs a retriever with a generator so that a language model can answer questions based on the information found in a document.

You:

Lab 24 — Build a Document-Based AI Assistant Supporting Multiple Uploaded Documents

Goal: Extend the assistant to accept several files at once, tag each chunk with its source filename as metadata, and answer questions across all of them — showing which document each answer came from.

Note: same fallback pattern as Lab 20 — update `file_paths` with your own Drive files if you have them; otherwise two short auto-generated sample documents are used so the lab still runs.

```
file_paths = [
    "/content/drive/MyDrive/College/CSA6502/1.pdf", # <-- optionally update this path
    "/content/drive/MyDrive/College/CSA6502/2.pdf", # <-- optionally update this path
]

multi_uploaded = {}
if drive_available:
    for p in file_paths:
        if os.path.exists(p):
            multi_uploaded[os.path.basename(p)] = p
        else:
            print(f"WARNING: file not found, skipping: {p}")

if not multi_uploaded:
    sample_a_path = "/content/sample_doc_a.txt"
    sample_b_path = "/content/sample_doc_b.txt"

    sample_a = """Vector embeddings are numeric representations of text produced by models such as sentence-transformers. Each piece of text is mapped to a point in a high-dimensional space so that pieces with similar meaning end up close together, even if they don't share any exact words. This is what lets a search for "how do plants make food" surface a document about photosynthesis: the two phrases are lexically different but semantically close. Cosine similarity is the standard way to measure how close two embeddings are, since it captures the angle between vectors rather than their raw magnitude."""

    sample_b = """LangChain is a framework for building applications on top of large language models by chaining together standard components: document loaders, text splitters, embedding models, vector stores, and the LLM itself. Its RetrievalQA and ConversationalRetrievalChain abstractions implement the RAG pattern out of the box, so a
```

developer can build a domain-specific chatbot in a few lines instead of writing the retrieval and prompting logic by hand. Adding a memory object lets the same chatbot handle multi-turn conversations, resolving follow-up questions using earlier turns."""

```
with open(sample_a_path, "w") as f:
    f.write(sample_a)
with open(sample_b_path, "w") as f:
    f.write(sample_b)

multi_uploaded = {"sample_doc_a.txt": sample_a_path, "sample_doc_b.txt": sample_b_path}
print("No Drive files found – using two auto-generated sample documents instead.")

print(f"{len(multi_uploaded)} file(s) ready: {list(multi_uploaded.keys())}")
```

2 file(s) ready: ['1.pdf', '2.pdf']

```
multi_collection = chroma_client.get_or_create_collection(
    name="multi_doc_assistant",
    embedding_function=sentence_transformer_ef,
    metadata={"hnsw:space": "cosine"},
)

splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)

all_ids = []
all_chunks = []
all_metadatas = []

for filename, filepath in multi_uploaded.items():
    text = load_text(filepath)
    file_chunks = splitter.split_text(text)
    for i, chunk in enumerate(file_chunks):
        all_chunks.append(chunk)
        all_ids.append(f"{filename}_{i}")
        all_metadatas.append({"source": filename})

multi_collection.upsert(documents=all_chunks, ids=all_ids, metadatas=all_metadatas)
print(f"Stored {len(all_chunks)} chunks from {len(multi_uploaded)} document(s).")
```

Stored 155 chunks from 2 document(s).

```
def multi_doc_assistant(question, k=3):
    results = multi_collection.query(query_texts=[question], n_results=k)
    retrieved_chunks = results["documents"][0]
    sources = [m["source"] for m in results["metadatas"][0]]
    context = "\n\n".join(
        f"[Source: {src}]\n{chunk}" for chunk, src in zip(retrieved_chunks, sources)
    )

    prompt = f"""Answer the question using ONLY the context below.
Mention which source document(s) the answer came from.
If the answer isn't in the context, say so.

Context:
{context}

Question: {question}
Answer: """
    return ask_llm(prompt), sources

question = "Compare the main topics across the uploaded documents."
answer, sources_used = multi_doc_assistant(question)
print("Answer:\n", answer)
print("\nSources used:", set(sources_used))
```

Answer:
Comparison of the main topics

Document	Main Topics (as described in the provided excerpts)
1.pdf	• Use of embeddings and semantic search • FAISS or ChromaDB as the vector database for Retrieval-Augmented Generation (RAG)
2.pdf	• Evaluation of low-resource language models (including comparative outcomes) • Analytical rigor: evidence-based evaluation

How the topics compare

- **Domain focus** - *1.pdf* is centered on the **technical implementation of a RAG system**, covering data preparation, vector storage, and retrieval.
- **Purpose** - *1.pdf* aims to **build and demonstrate** an end-to-end retrieval-augmented pipeline. *2.pdf* aims to **assess and document** the performance of different models.
- **Key components** - *1.pdf* discusses **embeddings, vector databases (FAISS/ChromaDB), query handling, and system architecture**. *2.pdf* focuses on **model evaluation metrics and comparative analysis**.

Thus, while both documents relate to natural-language processing, *1.pdf* concentrates on system construction and retrieval mechanics, whereas *2.pdf* emphasizes the evaluation and performance of language models.

Sources used: {'1.pdf', '2.pdf'}

Summary

Lab	Concept	Built on
15	Raw embeddings + cosine similarity	—
16	Reusable semantic search function	Lab 15
17	ChromaDB vector storage + query (cosine space)	Lab 16
18	Larger store + top-k retrieval function	Lab 17
19	RAG: retrieval + LLM answer generation	Lab 18
20	Full pipeline on a real/auto-generated file (load/chunk/embed/retrieve/generate)	Lab 19
21	Same pipeline via LangChain's <code>RetrievalQA</code>	Lab 20
22	Adds conversational memory via <code>ConversationalRetrievalChain</code>	Lab 21
23	Interactive loop wrapping the Lab 20 pipeline, with a non-interactive fallback	Lab 20
24	Multi-document assistant with per-chunk source metadata	Lab 20, 18